

APRA

APRA is a probabilistic decision engine that treats intelligence as repeated, evaluated exploration of possible explanations and actions. It generates many random hypotheses (guided randomness), scores them with likelihood and utility, resamples and refines them, and uses value of information to decide whether to gather more evidence or act. Repeating this loop makes randomness become reliably intelligent behavior.

1 — The basic probabilistic model

Outcomes and orders

Let there be K possible discrete outcomes o in $\mathcal{O}$.

APRA observes a sequence of orders (evidence channels) indexed by i in $\mathcal{I}$. Each order i produces an observable feature e_i (binary or real).

We have prior beliefs $P(o)$ and order models $P(e_i | o)$ (how likely order i reports feature e_i if the true outcome is o).

Generative model

The joint for an outcome o and observed orders $\mathbf{e}$ is:

$$P(o, \mathbf{e}) \\ ; = \\ ; P(o) \prod_{i \in \mathcal{I}} P(e_i | o).$$

Posterior over outcomes given observed orders

Given a subset (prefix) of observed orders obs :

$$P(o | \mathbf{e}_{\text{obs}}) \\ ; = \\ ; \frac{P(o) \prod_{i \in \text{obs}} P(e_i | o)}{\sum_{o'} P(o') \prod_{i \in \text{obs}} P(e_i | o')}.$$

Define unnormalized log-posterior:

$$\log \tilde{p}(o) \\ ; = \\ ; \log P(o) + \sum_{i \in \text{obs}} \log P(e_i | o),$$

then

$$P(o | \cdot) \\ ; = \\ ; \text{softmax} \\ \text{big}(\log \\ \text{tildep}(o) \\ \text{big}).$$

2 — Multi-order (1st to 10th degree) probabilities

You asked about higher orders (2nd, 3rd, ... up to 10th). APRA models them as additional orders \$i\$.

Practically:

- Order 1 is base evidence (fast, cheap).
- Order 2 is second-order evidence (depends on outcome but may be slower or more expensive).
- ... up to order L (e.g., $L = 10$).

APRA uses observed orders sequentially: each time a new order j is acquired, the posterior updates via the same formula above. Different orders have different cost, latency, and accuracy trade-offs and are used in VOI decisions (see below).

3 — Turning randomness into belief: sampling & particles (SMC)

APRA does not rely only on analytic posteriors. It uses particle sampling (Sequential Monte Carlo) to represent complex beliefs and to propose actions.

Particle representation

Maintain N particles $\{o^{(n)}, w^{(n)}\}_{n=1}^N$ with weights $w^{(n)}$ approximating the posterior:

$$w^{(n)} \propto P(o^{(n)}) \prod_{i \in \text{obs}} P(e_i | o^{(n)}).$$

Particle update (when new order arrives)

For new observed order j (order j):

1. For each particle n : update log-weight:

$$\log w^{(n)} \leftarrow \log w^{(n)} + \log P(e_j | o^{(n)}).$$

2. Normalize weights.
3. Compute ESS (effective sample size):

$$\text{ESS} = \frac{1}{\sum_n (w^{(n)})^2}.$$

If ESS is below a threshold, resample particles and optionally mutate.

This particle flow captures the randomness (diverse hypotheses) and focuses probability mass on likely outcomes.

4 — Decision-making: utilities and choosing actions

Associate actions $a \in \mathcal{A}$ and utilities $U(a, o)$ (reward of taking action a when true outcome is o).

Expected utility given posterior $\pi(o)$:

$$\begin{aligned} & \mathbb{E}[U(a)] \\ & ; = \\ & ; \\ & \sum_o \\ & p_i(o) \\ & , U(a, o). \end{aligned}$$

In a particle approximation:

$$\begin{aligned} & \mathbb{E}[U(a)] \\ & \approx \\ & ; \\ & \sum_{n=1}^N w^{(n)} \\ & , U \\ & \text{big}(a, o^{(n)}) \\ & \text{big}). \end{aligned}$$

5 — Value of Information (VOI): decide to gather more orders or act

APRA uses VOI to decide whether acquiring an additional order (costly test or measurement) is worth it.

For candidate order j , define:

- cost c_j (token cost, time, money)
- possible observations v

in

V_j with predictive probability $P(v)$

If observation v occurs, the posterior becomes $p_i(v|o)$ and the best achievable expected utility is:

KaTeX parse error: Got function '\ast' with no arguments as superscript at position 3: $U^{\ast}(v) = \dots$

Then expected post-observation utility is:

$$\begin{aligned}
 & \mathbb{E}_v \\
 & \Big[\\
 & \max_a \\
 & \sum_o \\
 & p_i(o)U(a, o) \\
 & \Big] \\
 & ; = \\
 & ; \\
 & \sum_v P(v) \\
 & \cdot \\
 & \max_a \\
 & \sum_o \\
 & p_i(o)U(a, o).
 \end{aligned}$$

Current best expected utility without new information:

KaTeX parse error: Got function '\|' with no arguments as superscript at position 3: U^\| \|ast_{\|text{now...}

Value of information:

KaTeX parse error: Got function '\|' with no arguments as superscript at position 91: ... \|Big]

\|;-
U^\| \|ast_{\|text{now...}

Net VOI (accounting for cost):

$$\begin{aligned}
 & \text{net}_V OI_j \\
 & ; = \\
 & ; \\
 & \text{VOI}_j - c_j.
 \end{aligned}$$

In practice compute VOI by:

- Analytic update if orders are simple; or
- Monte Carlo: sample hypothetical observations via particles and compute resulting best utility, then average.

Monte Carlo VOI pseudocode (sketch)

For $s = 1..S$:

1. Sample particle $o^{(n_s)}$ from current particles according to weights.
2. Sample hypothetical observation $v_s \sim P(v \mid o^{(n_s)})$.
3. Compute posterior $\pi_{v_s}(o)$ (e.g., by reweighting particles).
4. Compute best utility $U^*(v_s) = \max_a \sum_o \pi_{v_s}(o) U(a, o)$.

Then

KaTeX parse error: Got function '\|' with no arguments as superscript at position 55: ... \sum_{s=1}^S U^{\| \|}(v_s) \|;-
U...

6 — Planning under uncertainty (rollouts / path sampling)

For sequential decision problems, APRA uses Monte Carlo rollouts:

- For each candidate action (or action sequence), simulate future states using a world model (a stochastic model of how actions affect outcomes and future orders).
- For each simulated trajectory, accumulate utility and average across particles and rollouts to estimate expected return.
- Choose the best action or sequence.

This is standard Monte Carlo Tree Search / Model-Predictive-Control style planning but guided by APRA particles as initial state samples.

7 — Learning order models and priors (online updates)

APRA is adaptive: it refines order models $P(e_i \mid o)$ and priors $P(o)$ from experience.

For a simple binary feature case:

Maintain counts $N_{i,o}^{(1)}$ (times $e_i = 1$ when true outcome o observed) and $N_{i,o}^{(0)}$ (times $e_i = 0$ when true outcome o observed).

With a

*text*Beta(
alpha,

β) prior, the posterior mean estimate is:

$$\hat{P}(e_i = 1 | \text{data}) = \frac{\alpha + N_{i,o}^{(1)}}{\alpha + \beta + N_{i,o}^{(1)} + N_{i,o}^{(0)}}.$$

If outcomes are not observed directly, use EM-style responsibility-weighted updates using posterior weights:

- For each run where we later learn true outcome o , increment counts.
- If only partial feedback is available, use particle weights as soft counts.

8 — Putting it all together: APRA loop (pseudocode)

```

APRA_LOOP(task):
    initialize prior  $P(o)$  and order_models  $P(e_i | o)$ 
    initialize particles  $\{o^{(n)}, w^{(n)}\}$  from prior

    while True:
        # 1. propose actions from posterior
        for each action  $a$ :
            compute  $EU(a) = \sum_n w^{(n)} * U(a, o^{(n)})$ 
        best_now = max_a EU(a)

        # 2. compute VOI for candidate next orders  $j$ 
        for each candidate order  $j$ :
            VOI[j] = estimate_VOI(j) # Monte Carlo using particles
            net_VOI[j] = VOI[j] - cost_j

        # 3. decide: gather info or act
        if max_j net_VOI[j] > 0:
            j_star = argmax_j net_VOI[j]
            acquire observation  $e_{j\_star}$ 
            update particle weights with  $P(e_{j\_star} | o^{(n)})$ 
            normalize weights; resample if ESS low
            continue # loop and re-evaluate

```

else:

act: choose $a^* = \operatorname{argmax}_a EU(a)$ and execute

observe outcomes (feedback)

update order_models and priors with feedback

break or continue depending on task horizon

9 — Complexity (Big-O) and practical notes

Let:

- N = number of particles
- K = number of outcomes
- L = number of orders considered
- A = number of actions
- S = number of Monte Carlo samples for VOI or rollouts
- H = planning horizon (for rollouts)
- R = number of rollouts per action

Typical costs:

- Posterior weight update per new order: $O(N)$ (update weights).
- VOI Monte Carlo estimate: $O(S \cdot N)$ if naive, but S is usually small (50–200).
- Planning rollouts: $O(R \cdot H \cdot N)$ (simulate per particle).

Overall per decision: roughly

$$\begin{aligned}
&O \\
&\big(N + S \\
&\cdot N + R \\
&\cdot H \\
&\cdot N \\
&\big) \\
&; = \\
&; O \\
&\big(N(1 + S + RH) \\
&\big).
\end{aligned}$$

Choose N and S to trade accuracy versus latency. Typical starts: $N = 1\{, \}000$, $S = 50$, $R = 100$ for moderate quality.

10 — Why randomness (sampled hypotheses) $\rightarrow$ intelligence

Key idea: intelligence = good decisions under uncertainty. APRA converts randomness into intelligence by:

1. **Exploration** — drawing many hypotheses (particles) explores plausible explanations.
2. **Weighting** — likelihoods (order models) assign credit so improbable guesses are down-weighted.
3. **Aggregation** — expected utilities from the weighted ensemble give robust action choices.
4. **Information acquisition** — VOI tells when randomness is insufficient and further ordered evidence should be gathered.
5. **Refinement** — resampling and mutation focus computational effort on promising hypotheses (like attention).
6. **Learning** — feedback updates order models and priors, improving future randomness quality.

So the “perfect randomness” is not blind — it is directed random exploration: APRA uses prior, likelihood, and utility to shape randomness so it becomes a computationally efficient search for high-value actions.

11 — Example tiny numeric sketch (toy)

Outcomes: ['win','lose','draw'] $\rightarrow K = 3$

Priors:

- $P(\text{textwin}) = 0.5$
- $P(\text{textlose}) = 0.2$
- $P(\text{texdraw}) = 0.3$

Order-0 model:

- $p_0 = [0.8, 0.2, 0.5] = P(e_0 = 1 \mid \text{mido})$ for each outcome.

Observed $e_0 = 1$:

- Compute log-weights
 $\log P(o) + \log P(e_0 \mid \text{mido})$
- Exponentiate and normalize $\rightarrow$ posterior.

If utilities for `action0` are:

- $U(\text{textaction0}, \text{textwin}) = 30$
- $U(\text{textaction0}, \text{textlose}) = -50$
- $U(\text{textaction0}, \text{texdraw}) = 10$

Then

$$\begin{aligned} & \textit{textEU}(\textit{textaction0}) = [30, \\ & \quad , -50, \\ & \quad , 10] \\ & \quad \cdot \\ & \textit{textposterior}, \end{aligned}$$